

WORKING WITH OPENAI & ANTHROPIC REASONING MODELS

O'Reilly Live Training — Quick Reference Cheatsheet

Lucas Soares | GPT-5.5 · Claude Extended Thinking · DeepSeek R1 | 2026

1. WHAT IS A REASONING / THINKING MODEL?

Core Characteristics

- **Chain-of-Thought (CoT)** — spends time "thinking" before answering
- **Intermediate steps** — shows work, transparent & verifiable
- **Variable compute** — allocates more tokens for harder problems
- **RL-trained** — rewarded for step-by-step logical reasoning
- **Self-critique** — revises and verifies its own answers

System 1 vs System 2 Thinking

Traditional LLMs	Reasoning Models
Fast, intuitive (System 1)	Slow, methodical (System 2)
Pattern matching	Step-by-step analysis
Fixed forward passes	Variable compute time
General RLHF	RL with reasoning rewards

OpenAI o-series

- o1, o3, o4-mini
- Proprietary, API-only
- Reasoning effort: low / medium / high / xhigh
- Responses API (stateful)
- Vision support (o1+)

Anthropic Claude

- Claude 4 Opus/Sonnet
- Extended Thinking mode
- Budget tokens: 1K–32K+
- Interleaved thinking (Beta)
- Summarized thinking (Claude 4+)

DeepSeek R1

- 671B params, MIT license
- Open-source, self-hostable
- Distilled: 1.5B–70B params
- Strong math reasoning
- Hugging Face ecosystem

2. WHEN TO USE REASONING MODELS

Use Reasoning Models When...

- Accuracy & reliability > speed
- Complex, ambiguous, multi-step problems
- Explainable reasoning chains required
- Agentic planning & orchestration ("the planner")
- Finding relationships across large datasets
- Code review & architecture decisions
- LLM-as-judge evaluation tasks
- Navigating dense unstructured documents
- Visual reasoning over complex charts

Use Traditional LLMs When...

- Speed and cost are primary concerns
- Well-defined, straightforward tasks
- Task *execution* (not planning)
- Creative writing & conversational chat
- Content generation at volume
- Low-latency interactive applications
- Routing, summarization, triage
- o-series planner → GPT "doer" steps

3. REAL-WORLD USE CASES (WITH RESULTS)

Proven Enterprise Results

- **Tax research** — 4× performance boost (multi-doc reasoning, Blue J)
- **Code review** — 3× conversion increase (CodeRabbit)
- **LLM-as-judge** — F1 score 0.12 → 0.74 (Braintrust)
- **Image classification** — 88% vs 50% accuracy (SafetyKit)
- **Legal analysis** — 52% stronger on complex contracts (Hebbia)

Application Domains

- **Document Analysis** — contracts, financial statements, legal
- **Code** — debugging, architecture, one-shotting entire files
- **Decision Support** — financial analysis, risk assessment
- **Agentic Workflows** — o-series plans, GPT executes
- **Research** — hypothesis testing, paper synthesis
- **Visual Reasoning** — architectural drawings, dense charts

4. CHOOSING THE RIGHT MODEL — KEY CRITERIA

Selection Criteria & Trade-offs

Criterion	What to Evaluate	Trade-off
Reasoning Quality	CoT capabilities, logical consistency, benchmarks (MATH, GSM8K, MMLU)	Higher quality = higher cost & latency
Embedded Knowledge	Domain expertise, factual accuracy, knowledge cutoff date	Freshness vs. depth of specialization
Context Window	Document length, multi-turn conversation capacity	Larger window = more compute cost
Deployment	API-only vs. self-hostable; cloud vs. on-premises	Control & privacy vs. maintenance overhead
Cost	Cost per token, inference speed, hardware requirements	Performance vs. operational cost
Specialization	Code, math, multimodal, multilingual support	Generality vs. best-in-class for specific task

5. HOW TO PROMPT REASONING MODELS

OpenAI o-series (o1, o3, o4-mini)

- **Write briefs, not prompts** — 10× more context than you think
- **Focus on WHAT, not HOW** — let the model plan its approach
- **Keep it direct** — no "think step by step" needed (redundant)
- **Do NOT chain-of-thought prompt** — model reasons internally
- **Use delimiters** — markdown, XML tags, section titles
- **Zero-shot first**, then few-shot if needed
- **Define success criteria** — let model self-evaluate
- **Developer messages** (not system messages) for o1-2024-12-17+
- **Reasoning effort API**: `none` / `low` / `medium` / `high` / `xhigh`
- **Responses API** with `store=true` for multi-turn + tool calls

Anthropic Claude Extended Thinking

- **Budget tokens** — control depth: 1K (quick) → 32K+ (deep)
- **Start minimal** — don't say "think step by step" (redundant)
- **Streaming required** for large outputs (improves UX)
- **Tool use**: set `tool_choice: auto`
- **Interleaved thinking** — Beta, Claude 4 models only
- **Summarized thinking** — Claude 4+ returns summary; billed for full tokens
- **Show progress** — stream thinking blocks to reduce perceived latency
- **Complex tasks** — increase budget progressively, don't start at max

Key Mental Model: o1 is a "Report Generator", not a Chat Model

Treat it like email, not texting. Give full context upfront (company background, schemas, code, constraints). Ask for one specific output. Define what "good" looks like. It will one-shot entire files — but won't iteratively chat its way to a solution.

6. API QUICK REFERENCE

OpenAI — Responses API (o3)

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="o3",
    reasoning={"effort": "high"},
    input=[{
        "role": "user",
        "content": "Your detailed
brief here..."
    }],
    store=True # enable multi-turn
    reasoning
)
print(response.output_text)
```

Anthropic — Extended Thinking (Claude)

```
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=16000,
    thinking={
        "type": "enabled",
        "budget_tokens": 10000
    },
    messages=[{
        "role": "user",
        "content": "Your problem
here..."
    }]
)
# Access thinking: response.content[0]
# Access answer: response.content[1]
```

OpenAI — Streaming Reasoning

```
with client.responses.stream(
    model="o3",
    reasoning={"effort": "medium"},
    input=[{"role": "user", "content":
brief}]
) as stream:
    for event in stream:
        # Handle text deltas in real-
time
        if event.type ==
"response.text.delta":
            print(event.delta, end="",
flush=True)
```

Anthropic — Streaming Extended Thinking

```
with client.messages.stream(
    model="claude-opus-4-8",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tok
ens": 8000},
    messages=[{"role": "user", "content": pro
blem}]
) as stream:
    for event in stream:
        if event.type ==
"content_block_delta":
            # thinking_delta or
text_delta
            print(event.delta)
```

7. DEEPSEEK R1 VS OPENAI O1 — COMPARISON

Model Comparison

Dimension	DeepSeek R1	OpenAI o1/o3
License	Open-source (MIT)	Proprietary
Size	671B parameters (full)	Undisclosed
Access	API + self-host + Hugging Face	OpenAI API only
Cost	Free (self-host) / API pricing	Pay-per-token
Math	Particularly strong	Excellent (occasional gaps)
Customization	Full: quantize, fine-tune, prune	Limited
Deployment	Cloud + on-premises + edge	Cloud only
Ecosystem	Growing OSS community	Mature commercial support

Distilled Models (DeepSeek-R1-Distill)

Available in 1.5B → 70B parameter sizes. Achieve 80–95% of teacher model performance.

Llama-70B distill: 94.5% on MATH-500, 86.7% on AIME 2024. **Qwen-32B distill** outperforms OpenAI-o1-mini across multiple benchmarks. Use when: resource-constrained, edge deployment, latency-sensitive, or cost-sensitive applications.

8. LIMITATIONS & MITIGATION STRATEGIES

Known Limitations

- **Latency** — significantly longer response times
- **Cost** — more reasoning tokens = higher cost
- **Domain brittleness** — inconsistent across subject areas
- **Over-confidence** — may present flawed reasoning confidently
- **Knowledge cutoff** — bounded by training data
- **Not for** — creative writing in specific voices
- **Not for** — building entire apps in one shot
- **Context limits** — struggles with very long reasoning chains

Mitigation Strategies

- **Hybrid approach** — reasoning model plans, GPT executes
- **Fallback mechanisms** — alternate path when reasoning fails
- **Human-in-the-loop** — critical decisions need human review
- **Progressive UI** — stream thinking steps to reduce perceived latency
- **Model ensembles** — consensus from multiple models
- **Domain testing** — benchmark on your specific task type
- **Distilled models** — for cost/latency-sensitive deployments

9. COURSE NOTEBOOKS

analytical-framework.ipynb

- LLM-as-a-Judge framework
- Compare GPT-4o, GPT-5.5, Claude Sonnet 4.6
- Pydantic structured evaluation
- Cost analysis with 2026 pricing
- PDF-to-markdown conversion

openai-thinking-parameters.ipynb

- GPT-5.5 Responses API deep-dive
- Reasoning effort levels (none→xhigh)
- Verbosity control (independent)
- Tool use + web search integration
- Structured outputs with Pydantic

anthropic-extended-thinking.ipynb

- Claude extended thinking setup
- Budget tokens tuning (1K–32K+)
- Streaming implementation
- Interleaved thinking (Beta)
- Tool use with extended thinking

10. QUICK DECISION FRAMEWORK

Model Selection Tree

Need reasoning model?

└ Speed > accuracy? → GPT-4o / traditional LLM

└ Complex + ambiguous problem?

 └ Need on-premises? → DeepSeek R1 (self-host)

 └ Cost-sensitive? → Distilled model (32B)

 └ Best accuracy? → OpenAI o3 or Claude Opus

 └ Vision needed? → OpenAI o1/o3

└ Agentic workflow?

 └ Planner role → o-series / Claude Extended Thinking

 └ Executor role → GPT-4o / Claude Sonnet (fast)

Reasoning Effort Guidelines

Effort	Use When
low	Well-defined, structured tasks
medium	Most standard complex problems
high	Multi-doc analysis, hard math
xhigh	Maximum accuracy critical tasks

Claude budget tokens:

1K–2K = quick reasoning

8K–16K = standard complex tasks

32K+ = deep analysis, long documents

KEY TAKEAWAYS

- **Reasoning models** trade speed for accuracy via internal CoT
- **Write briefs** for o-series — 10× context, clear goals, one output
- **Budget tokens** control Claude thinking depth (start minimal)
- **Don't say** "think step by step" — redundant for both APIs
- **DeepSeek R1** is open-source, self-hostable, MIT licensed
- **Distilled models** reach 80–95% of teacher model performance
- **Planner + doer pattern** — o-series plans, GPT executes
- **Stream responses** to manage latency expectations in production
- **Hybrid fallbacks** — design for when reasoning fails or times out
- **Benchmark on YOUR domain** — performance varies significantly by task type